UNIT-IV
PUSHDOWN AUTOMATA

Introduction of Pushdown Automata

We have already discussed finite automata. But finite automata can be used to accept only regular languages. Pushdown Automata is a finite automata with extra memory called stack which helps Pushdown automata to recognize Context Free Languages.

Pushdown Automata

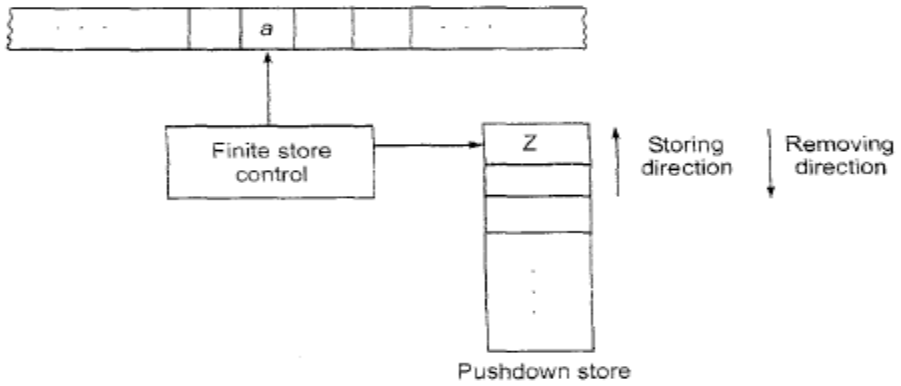
A Pushdown Automata (PDA) can be defined as : It is a 7-tupe

M=(Q, ∑, Γ, δ,q0,Z,F)

- Q is the set of states
- ∑is the set of input symbols
- Γ is the set of pushdown symbols (which can be pushed and popped from the stack)
- q0 is the initial state
- Z is the initial pushdown symbol (which is initially present in the stack)
- F is the set of final states
- δ is a transition function that maps Q x {Σ ∪ ∈} x Γ →Q x Γ*. In a given state, the PDA will read the input symbol and stack symbol (top of the stack) move to a new state, and change the symbol of the stack.

Model of PDA:

- Pushdown automaton has a read-only input tape, an input alphabet a finite state control, a set of final states, and an initial state as in the case of an FA.
- In addition to these, it has a stack called the pushdown store. It is a read-write pushdown store as we can add elements to PDS or remove elements from PDS.
- A finite automaton is in some state and on reading, an input symbol moves to a new state.
- The pushdown automaton is also in some state and on reading an input symbol and the topmost symbol in PDS, it moves to a new state and writes (adds) a string of symbols in PDS.



Graphical Notation for Pushdown Automata Now, let's talk about how we can draw a Pushdown Automata. It's often easier to understand when we can see it!

Basic Elements

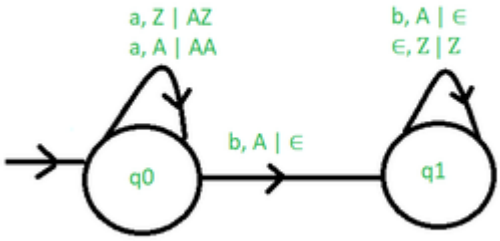
- States − We draw these as circles.
- Start State − This is a circle with an arrow pointing to it.
- Final State − We use a double circle for this.
- Transitions − These are arrows between states.

Construct Graphical notation for the following PDA transitions:

M=(Q, ∑, Γ, δ,q0,Z,F),

PDA transitions are
δ(q0, a, Z) = { (q0, AZ) }
δ(q0, a, A) = { (q0, AA) }
δ(q0, b, A) = { (q1, ∈) }
δ(q1, b, A) = { (q1, ∈) }
δ(q1, ∈, Z) = { (q1, ∈) }

Push Down Automata State Diagram



Instantaneous Description (ID)

Instantaneous Description (ID) is an informal notation of how a PDA "computes" an input string and makes a decision whether that string is accepted or rejected.
An ID is a triple (q, w, α), where:
1. q is the current state.
2. w is the remaining input.
3.α is the stack contents, top at the left.

Accepted Languages:

For PDA $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ we define $L(M)$, the *language accepted by final state*, to be

$$\{w \mid (q_0, w, Z_0) \vdash^* (p, \epsilon, \gamma) \text{ for some } p \text{ in } F \text{ and } \gamma \text{ in } \Gamma^*\}.$$

We define $N(M)$, the *language accepted by empty stack* (or *null stack*) to be

$$\{w \mid (q_0, w, Z_0) \vdash^* (p, \epsilon, \epsilon) \text{ for some } p \text{ in } Q\}.$$

When acceptance is by empty stack, the set of final states is irrelevant, and, in this case, we usually let the set of final states be the empty set.

Design of PDA:

***Example1*:**

Construct a PDA for language L = {$0^n1^m2^m3^n$ | n>=1, m>=1}

Approach used in this PDA -

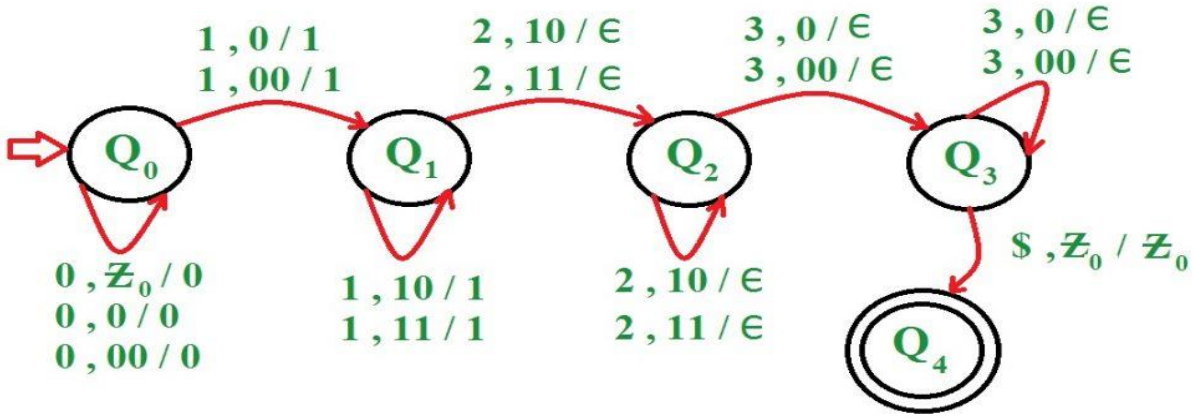
First 0's are pushed into stack. Then 1's are pushed into stack.
Then for every 2 as input a 1 is popped out of stack. If some 2's are still left and top of stack is a 0 then string is not accepted by the PDA. Thereafter if 2's are finished and top of stack is a 0 then for every 3 as input equal number of 0's are popped out of stack. If string is finished and stack is empty then string is accepted by the PDA otherwise not accepted.

Step-1: On receiving 0 push it onto stack. On receiving 1, push it onto stack and goto next state

Step-2: On receiving 1 push it onto stack. On receiving 2, pop 1 from stack and goto next state

Step-3: On receiving 2 pop 1 from stack. If all the 1's have been popped out of stack and now receive 3 then pop a 0 from stack and goto next state

Step-4: On receiving 3 pop 0 from stack. If input is finished and stack is empty then goto last state and string is accepted



Design a PDA that accepts {ww^R | w in (0+1)*}

L = { ε, 0, 1, 00, 11, 0110, 1001, ...}

Let M= (Q, ∑, Γ, δ, q_0, Z_0, F) be the PDA

Consider M = ({q1, q2}, {0, 1}, {0, 1, Z_0}, δ, q_1, Z_0, Ø)

δ(q1,0,Z_0)={(q1,0Z_0)}
δ(q1,1,Z_0)={(q1,1Z_0)}
δ(q1,0,0)={(q1,00),(q2, ε)}
δ(q1,1,0)={(q1,10)}
δ(q1,0,1)={(q1,01)}
δ(q1,1,1)={(q1,11),(q2, ε)}
δ(q2,0,0)={(q2, ε)}
δ(q2,1,1)={(q2, ε)}
δ(q1,ε,Z_0)={(q2, ε }
δ(q2,ε,Z_0)={(q2, ε)}

Deterministic PDA:

The PDA is deterministic in the sense that at most one move is possible from any ID.

Formally we say a PDA M is deterministic if:

1) for each q in Q and Z in Γ, whenever $\delta(q, \epsilon, Z)$ is nonempty, then $\delta(q, a, Z)$ is empty for all a in Σ;
2) for no q in Q, Z in Γ, and a in $\Sigma \cup \{\epsilon\}$ does $\delta(q, a, Z)$ contain more than one element.

Equivalence of PDA's and CFL's :

CFG to PDA Conversion

If L is a context-free language, then there exists a PDA M such that L = N(M).

Procedure:

Let L=L(G), where G=(V_N, Σ,P,S) is a context free grammar.
We construct a PDA M as
M= ((q), Σ, V_N U Σ, Z_0,q, δ, Ø)
Where δ is defined by the following rules:
R1: δ (q, ε,A) = {(q, α) | A -> α is in P}
R2: δ (q,a,a) = { (q, ε)} for every a in Σ.
Example:
Construct a pda M equivalent to the following context free grammar:
S->0BB
B->0S|1S|0.
Test whether 010^4 is in N(M).
Solution:
Define pda A as follows:
A=(({q},{0,1},{S,B,0,1}, δ,q, Z_0, Ø)
δ is defined by the following rules:
R_1: δ(q, ε, S)={(q,0BB)}
R_2: δ(q, ε, B)={(q,0S),(q,0S),(q,0)}
R_3: δ(q, 0,0)={(q, ε)}
R_4: δ(q, 1,1)={(q, ε)}

String Checking

(q,010^4,S)	
⊢ (q, 010^4, 0BB)	by Rule R_1
⊢ (q, 10^4, BB)	by Rule R_3
⊢ (q, 10^4,1SB)	by Rule R_2 since (q, 1S) ∈ α(q, ∧, B)
⊢ (q, 0^4, SB)	by Rule R_4
⊢ (q, 0^4, 0BBB)	by Rule R_1
⊢ (q, 0^3,BBB)	by Rule R_3
$\vdash^*$ (q, 0^3,000)	by Rule R_2 since (q,0) ∈ α(q, ∧, B)
$\vdash^*$ (q, ε , ε)	by Rule R_3

Conversion of PDA to CFG:
If $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ is a PDA, then there exists a context-free grammar G such that L(G) = N(A).

Step 1 (Construction of G). We define $G = (V_N, \Sigma, P, S)$, where

$$V_N = \{S\} \cup \{[q, Z, q'] \mid q, q' \in Q, Z \in \Gamma\}$$

i.e. any element of V_N is either the new symbol S acting as the start symbol for G or an ordered triple whose first and third elements are states and the second element is a pushdown symbol.

The productions in P are induced by moves of pda as follows:

R_1: S-productions are given by $S \to [q_0, Z_0, q]$ for every q in Q.

R_2: Each move erasing a pushdown symbol given by $(q', \Lambda) \in \delta(q, a, Z)$ induces the production $[q, Z, q'] \to a$.

R_3: Each move not erasing a pushdown symbol given by $(q_1, Z_1Z_2 \ldots Z_m) \in \delta(q, a, Z)$ induces many productions of the form

$$[q, Z, q'] \to a[q_1, Z_1, q_2][q_2, Z_2, q_3] \ldots [q_m, Z_m, q']$$

where each of the states $q', q_2, \ldots, q_m$ can be any state in Q. Each move yields many productions because of R_3. We apply this construction to an example before proving that $L(G) = N(A)$.

Example:

$$M = (\{q_0, q_1\}, \{0, 1\}, \{X, Z_0\}, \delta, q_0, Z_0, \varnothing),$$

where δ is given by

$$\delta(q_0, 0, Z_0) = \{(q_0, XZ_0)\}, \qquad \delta(q_1, 1, X) = \{(q_1, \epsilon)\},$$
$$\delta(q_0, 0, X) = \{(q_0, XX)\}, \qquad \delta(q_1, \epsilon, X) = \{(q_1, \epsilon)\},$$
$$\delta(q_0, 1, X) = \{(q_1, \epsilon)\}, \qquad \delta(q_1, \epsilon, Z_0) = \{(q_1, \epsilon)\}.$$

To construct a CFG $G = (V, T, P, S)$ generating $N(M)$ let

$$V = \{S, [q_0, X, q_0], [q_0, X, q_1], [q_1, X, q_0], [q_1, X, q_1],$$
$$[q_0, Z_0, q_0], [q_0, Z_0, q_1], [q_1, Z_0, q_0], [q_1, Z_0, q_1]\}$$

and $T = \{0, 1\}$. To construct the set of productions easily, we must realize that some variables may not appear in any derivation starting from the symbol S.

Add productions only for those variables that appear on the right of some production already in the set. The productions for S are

$$S \to [q_0, Z_0, q_0]$$
$$S \to [q_0, Z_0, q_1]$$

Next we add productions for the variable $[q_0, Z_0, q_0]$. These are

$$[q_0, Z_0, q_0] \to 0[q_0, X, q_0][q_0, Z_0, q_0]$$
$$[q_0, Z_0, q_0] \to 0[q_0, X, q_1][q_1, Z_0, q_0]$$

These productions are required by $\delta(q_0, 0, Z_0) = \{(q_0, XZ_0)\}$. Next, the productions for $[q_0, Z_0, q_1]$ are

$$[q_0, Z_0, q_1] \to 0[q_0, X, q_0][q_0, Z_0, q_1]$$
$$[q_0, Z_0, q_1] \to 0[q_0, X, q_1][q_1, Z_0, q_1]$$

These are also required by $\delta(q_0, 0, Z_0) = \{(q_0, XZ_0)\}$. The productions for the remaining variables and the relevant moves of the PDA are:

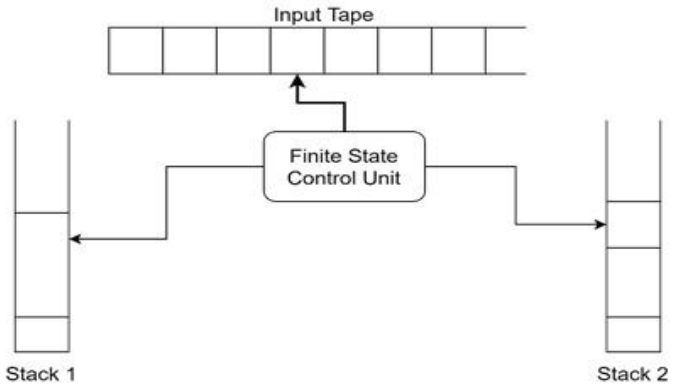
1) $[q_0, X, q_0] \to 0[q_0, X, q_0][q_0, X, q_0]$
   $[q_0, X, q_0] \to 0[q_0, X, q_1][q_1, X, q_0]$
   $[q_0, X, q_1] \to 0[q_0, X, q_0][q_0, X, q_1]$
   $[q_0, X, q_1] \to 0[q_0, X, q_1][q_1, X, q_1]$
   since $\delta(q_0, 0, X) = \{(q_0, XX)\}$.
2) $[q_0, X, q_1] \to 1$ since $\delta(q_0, 1, X) = \{(q_1, \epsilon)\}$.
3) $[q_1, Z_0, q_1] \to \epsilon$ since $\delta(q_1, \epsilon, Z_0) = \{(q_1, \epsilon)\}$.
4) $[q_1, X, q_1] \to \epsilon$ since $\delta(q_1, \epsilon, X) = \{(q_1, \epsilon)\}$.
5) $[q_1, X, q_1] \to 1$ since $\delta(q_1, 1, X) = \{(q_1, \epsilon)\}$.

Two-stack Pushdown Automata and Turing Machine

- A traditional Pushdown Automaton has one stack, but there could be some variations. We can also have a two-stack pushdown automaton. Both are powerful computational models, but they differ in their structure and capabilities.

Basics of Two-stack Pushdown Automata

- A Pushdown Automaton is a computational model similar to a finite automaton but with an added stack. The stack provides the machine with additional memory. These are used for recognizing context-free languages.
- However, a standard PDA has limitations, particularly when it comes to languages that require more complex memory management. We can put another stack to make two-stack PDA.
- Take a look at the following block diagram −



Key Features of Two-stack Pushdown Automata

Following are some of the key features of two-stack pushdown automata −

- **Two Stacks** − Unlike a standard PDA, which has only one stack, a two-stack PDA has two stacks. This dual-stack setup allows it to simulate a Turing machine, making it more powerful than a single-stack PDA.
- **Memory Management** − With two stacks, the PDA can store and manage more complex information, enabling it to recognize languages that a standard PDA cannot.
- **Deterministic and Non-deterministic** − A two-stack PDA can be either deterministic or non-deterministic, but it is the non-deterministic version that fully captures the power of a Turing machine.

How Does a Two-Stack Pushdown Automata Work?

In a two-stack PDA, the machine uses one stack to push and pop symbols while the other stack performs similar operations. The interaction between these two stacks enables the PDA to keep track of more complex structures, such as nested sequences or multiple dependencies, which are common in context-sensitive languages.

Comparison with Turing Machine

- The Turing Machine is most powerful computational model that can simulate any algorithm. It consists of an infinite tape, a tape head that reads and writes symbols on the tape, and a finite state control that dictates the machine's operations based on the current state and the symbol under the tape head.

Relationship between Two-stack PDA and Turing Machine

A Two-Stack PDA and Turing Machines are computationally equivalent that we can prove. A PDA is similar to FSM, but it has an additional memory, a stack, to help it recognize strings. If a PDA has more than one stack, we can call it a Turing machine.

Simulation − A two-stack PDA can simulate a Turing machine by using one stack to represent the portion of the tape to the left of the head and the other stack to represent the portion to the right.

Components of Two-stack PDA

A two-stack PDA is defined by a 9-tuple: $M=(Q,\Sigma,\Gamma,\Gamma',\delta,q0,z1,z2,F)$

- **Q** − All the states the PDA can be in
- **Σ** − The input alphabet (symbols it can read)
- **Γ** − The alphabet for the first stack
- **Γ'** − The alphabet for the second stack
- **δ** − Rules for moving between states and using the stacks
- **q0** − The starting state
- **z1** − The bottom symbol for the first stack
- **z2** − The bottom symbol for the second stack
- **F** − The final or accepting states

Example: Recognizing $\{a^nb^nc^n|n\geq 0\}$

Let's see how a two-stack PDA can handle the language $a^nb^nc^n|n\geq 0\}$. This language has equal numbers of a's, b's, and c's in that order. Which is not solvable by one stack PDA, but can be solvable by Turing Machines.

Strategy − Here's how we'll use our two stacks −

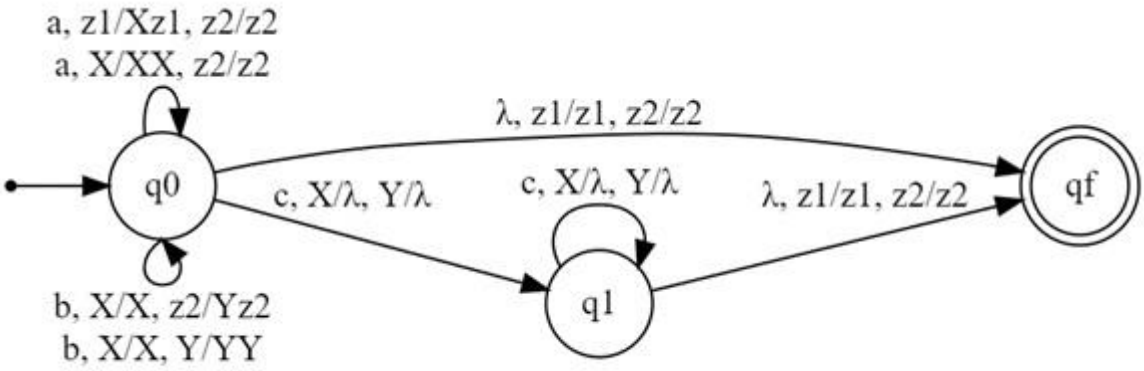
- While reading 'a', we'll push 'X' into stack 1.
- While reading 'b', we'll push 'Y' into stack 2.
- While reading 'c', we'll pop 'X' from stack 1 and 'Y' from stack 2.
- If both stacks are empty at the end, we accept the string.

Transition Functions − Here are the transition functions −

$\delta(q0,\lambda,z1,z2)\rightarrow(qf,z1,z2)$

$\delta(q0,a,z1,z2)\rightarrow(q0,Xz1,z2)$

$\delta(q0,a,X,z2) \rightarrow (q0,XX,z2)$
$\delta(q0,b,X,z2) \rightarrow (q0,X,Yz2)$
$\delta(q0,b,X,Y) \rightarrow (q0,X,YY)$
$\delta(q0,c,X,Y) \rightarrow (q1,\lambda,\lambda)$
$\delta(q1,c,X,Y) \rightarrow (q1,\lambda,\lambda)$
$\delta(q1,\lambda,z1,z2) \rightarrow (qf,z1,z2)$



Explanation of Transitions

- The first line handles the case of an empty input string.
- The next two lines handle reading 'a' and pushing 'X' to stack 1.
- The fourth and fifth lines handle reading 'b' and pushing 'Y' to stack 2.
- The sixth and seventh lines handle reading 'c' and popping from both stacks.
- The last line checks if we have finished the string with empty stacks.

Applications of PDA:

Pushdown Automata are utilized in:

- **Syntax Analysis in Compilers**: Helps parse programming language structures by using a stack to manage nested elements (e.g., parentheses).
- **Stack-Based Applications**: Used in scenarios where operations depend on the last inserted element, like evaluating arithmetic expressions.
- **Tower of Hanoi Problem**: Solves problems involving recursive and stack-based solutions.
- **Network Protocols**: PDA can validate message formats and enforce structured communication.
- **Natural Language Processing**: Used in tasks such as parsing sentences and generating syntax trees.
- **Cryptography**: Helps in designing algorithms for encryption and decryption.
- **Automatic Theorem Proving**: PDA is applied to verify the correctness of software models and systems.

Assignment-Cum-Tutorial Questions

A. Questions testing the understanding / remembering level of students

I) Objective Questions

1. Definition: What distinguishes a Pushdown Automaton (PDA) from a Finite Automaton (FA)? [ ]

A. PDA uses multiple input tapes
B. PDA uses a stack as auxiliary memory
C. PDA accepts only regular languages
D. PDA can only be deterministic

2. Model: Which of the following components is NOT part of a PDA? [ ]

A. Input Alphabet
B. Stack Alphabet
C. Stack Pointer
D. Transition Function

3. Graphical Notation: In PDA state diagrams, what does the label on a transition typically represent? [ ]

A. State name
B. (Input symbol, Stack top / Stack action)
C. (Stack top, Input symbol / New state)
D. (Current state / Stack action)

4. Instantaneous Description: Which of the following best represents an Instantaneous Description (ID) of a PDA? [ ]

A. A set of production rules
B. A triple (q, w, γ) where q is the state, w is the input, and γ is the stack contents
C. A binary tree
D. A tuple ($Q, \Sigma, \Gamma, \delta, q_0, Z_0, F$)

5. Language Acceptance: A PDA can accept a language by which of the following methods? [ ]

A. Final state only
B. Empty stack only
C. Either final state or empty stack
D. Both final state and empty stack must be reached

6. Design of PDA: Which language is accepted by the following PDA design principle: push on input 'a', pop on input 'b'? [ ]

A. $a^n b^m$ where $n \neq m$
B. $a^n b^n$ where $n \geq 0$
C. $a^n b^n c^n$
D. $b^n a^n$

7. Determinism: Which of the following is TRUE for a deterministic PDA (DPDA)? ***[ ]***

A. DPDA can accept all context-free languages
B. DPDA and NDPDA are equivalent in power
C. DPDA has at most one possible transition for each input and stack combination
D. DPDA can simulate a Turing Machine

8. Equivalence: Pushdown Automata (PDA) are equivalent in power to which formal grammar class? ***[ ]***

A. Regular Grammars
B. Context-Free Grammars
C. Context-Sensitive Grammars
D. Unrestricted Grammars

9. Example: Which of the following languages is NOT context-free and thus cannot be accepted by a single-stack PDA? ***[ ]***

A. $a^n b^n$
B. $a^n b^n c^n$
C. (a|b)*
D. Palindromes over {a, b}

10. Two-Stack PDA: What is the computational power of a PDA with two stacks? ***[ ]***

A. Equivalent to Regular Automata
B. Equivalent to single-stack PDA
C. Equivalent to Turing Machines
D. Less powerful than Turing Machines

11. Application: Which of the following is a practical application of PDAs? ***[ ]***

A. String sorting
B. Lexical analysis
C. Parsing in compilers
D. Deadlock detection

II) Descriptive questions

Q1: Define a Pushdown Automaton (PDA). How does it differ from a Finite Automaton (FA)?

Q2: What is the formal definition (7-tuple) of a PDA? Explain each component with an example.

Q3: Explain the model of a Pushdown Automaton. What role does the stack play in computation?

Q4: Illustrate with an example how a PDA processes an input string using its stack.

Q5: Describe how transitions are represented in the graphical (state diagram) notation of a PDA.

Q6: Draw a PDA that accepts the language $L = \{ a^n b^n \mid n \geq 0 \}$ and explain its transitions.

Q7: What is an Instantaneous Description (ID) in the context of PDAs? What does it represent?

Q8: Describe how IDs are used to trace the computation steps of a PDA on a given input.

Q9: How does a PDA accept a language? Explain the two modes of acceptance: by **final state** and by **empty stack**.

Q10: Give an example of a PDA that accepts a language by empty stack, and show how it processes a sample input.

Q11: Explain the general approach to designing a PDA for a language of the form $L = \{ a^n b^n \mid n \geq 0 \}$.

Q12: Design a PDA for the language $L = \{ w \in \{a, b\}^* \mid w \text{ is a palindrome} \}$ and explain the logic.

Q13: Differentiate between a **Deterministic PDA (DPDA)** and a **Non-deterministic PDA (NPDA)**.

Q14: Why are DPDAs less powerful than NPDAs? Give an example of a language accepted by NPDA but not by any DPDA.

Q15: Prove that every Context-Free Grammar (CFG) has an equivalent PDA.

Q16: Convert the following CFG into an equivalent PDA: $S \rightarrow aSb \mid \varepsilon$

Q17: Give two examples of languages that are accepted by a PDA and explain why.

Q18: Explain why the language $L = \{ a^n b^n c^n \mid n \geq 0 \}$ cannot be accepted by any PDA.

Q19: What is a Two-Stack PDA? How is it more powerful than a single-stack PDA?

Q20: Explain how a two-stack PDA can simulate a Turing Machine.

Q21: What are the real-world applications of Pushdown Automata? Explain at least two in detail.

Q22: Describe how PDAs are used in parsing and syntax analysis in compilers.

B. Question testing the ability of students in applying the concepts.

1. $L=\{ 0^n 1^{2n} \mid n>=1\}$ is [ ]

   a) regular b) context-free but not regular c) context-free but regular

   d) None

2. Recognize the language accepted by the PDA with the following moves [ ]

   $\delta (q_0,a,Z_0)=(q_0,aZ_0)$, $\delta (q_0,a,a)=(q_0,aa)$

   $\delta (q_0,b,a)=(q_1, \varepsilon)$, $\delta (q_1,b,a)=(q_1, \varepsilon)$

   $\delta (q_1,c,Z_0)=(q_2, Z_0)$, $\delta (q_2,c,Z_0)=(q_2, Z_0)$

   a) $L=\{a^n b^n c^n \mid n, m>=1\}$ b) $L=\{a^n b^n c^m \mid n, m>=1\}$

   c) $L=\{a^m b^n c^n \mid n, m>=1\}$ d) $L=\{a^m b^n c^m \mid n, m>=1\}$

3. The grammar G1: S -> 0S0| 1S1 | 0|1| ε and the grammar [ ]

   G2: is S -> as |asb| X, X -> Xa | a.

   Which is the correct statement?

   a) G1 is ambiguous, G2 is unambiguous b) G1 is unambiguous, G2 is ambiguous

   c) Both G1 and G2 are ambiguous d) Both G1 and G2 are unambiguous

II) Problems

1. Construct the PDA that accepts the language $L=\{a^n b^m c^n \mid n, m >=1\}$.

2. Design PDA for the language L= {ww^r |w belongs {a,b}*}.
3. Consider the following grammar. S→AA/a A→SA/b and construct a PDA M such that the language generated by M and G are equivalent.
4. Construct a context-free grammar G which accepts N(A),where A=({q0,q1},{a,b},{Z0,Z1}, δ,q0,Z0, Ø) and δ is given by

$$\delta(q_0,\ b,\ Z_0) = \{(q_0,\ ZZ_0)\}$$
$$\delta(q_0,\ \Lambda,\ Z_0) = \{(q_0,\ \Lambda)\}$$
$$\delta(q_0,\ b,\ Z) = \{(q_0,\ ZZ)\}$$
$$\delta(q_0,\ a,\ Z) = \{(q_1,\ Z)\}$$
$$\delta(q_1,\ b,\ Z) = \{(q_1,\ \Lambda)\}$$
$$\delta(q_1,\ a,\ Z_0) = \{(q_0,\ Z_0)\}$$

5. Prove that the language L = {www |w ∈ {a, b} ∗} is not context free.

6. ***Questions testing the analyzing /evaluating ability of students.***
7. Analyze the language accepted by this PDA by empty store?

Let $M = (\{q_0,q_1\},\{0,1\},\{z_0,X\},\delta,q_0,z_0,\phi)$ be a Pushdown automation where δ is given by

$$\delta(q_0,1,z_0) = \{(q_0,Xz_0)\}$$
$$\delta(q_0,\epsilon,z_0) = \{(q_0,\epsilon)\}$$
$$\delta(q_0,1,X) = \{(q_0,XX)\}$$
$$\delta(q_1,1,X) = \{(q_1,\epsilon)\}$$
$$\delta(q_0,0,X) = \{(q_1,X)\}$$
$$\delta(q_0,0,z_0) = \{(q_0,z_0)\}$$

C. GATE/NET/SLET

1. Consider the languages: [ ]

L_1 ={ww^R | w belongs {0,1}*}

L_2 ={$w\#w^R$ | w belongs {0,1}*},where # is a special symbol

L_3 ={ww | w belongs {0,1}*}

Which one of the following is TRUE? [Gate 2005]

a) L_1 is a deterministic CFL
b) L_2 is a deterministic CFL
c) L_3 is a CFL,but not a deterministic CFL
d) L_3 is a deterministic CFL

2. Consider the following statements about the context free grammar [ ] [GATE-CS-2006]

G = {S → SS, S → ab, S → ba, S → E}

I. G is ambiguous

II. G produces all strings with equal number of a's and b's

III. G can be accepted by a deterministic PDA.

Which combination below expresses all the true statements about G?

a) Ionly **b)** I and III only

c) I and II only **d)** I, II and III

3. If L1 is context free language and L2 is a regular language which of the following is/are false? [Gate 1999]

a) L1-L2 is not context free b) L1 ∩ L2 is context free
c) ~L1 d) ~L2 [ ]

4. Let L_D be the set of all languages accepted by a PDA by final state and L_E the set of all languages accepted by empty stack.Which of the following is true? [Gate 1999] [ ]

a) $L_D = L_E$ b) $L_D \subset L_E$ c) $L_E \supset L_D$ d) None of the above

5. Context-free languages are closed under: [ ]

a) Union, Intersection b) Union, Kleene closure
c) Intersection, complement d) Complement, Kleene closure [Gate1998]

6. Identify the language generated by the following grammar, Where S is the start variable. [Gate 2017]

S->XY
X->aX |a
Y->aYb |epsilon

A) $\{a^m b^n \mid m>=n, n>0\}$ B) $\{a^m b^n \mid m>=n, n>=0\}$ [ ]

C) $\{a^m b^n \mid m>n, n>=0\}$ D) $\{a^m b^n \mid m>n, n>0\}$